

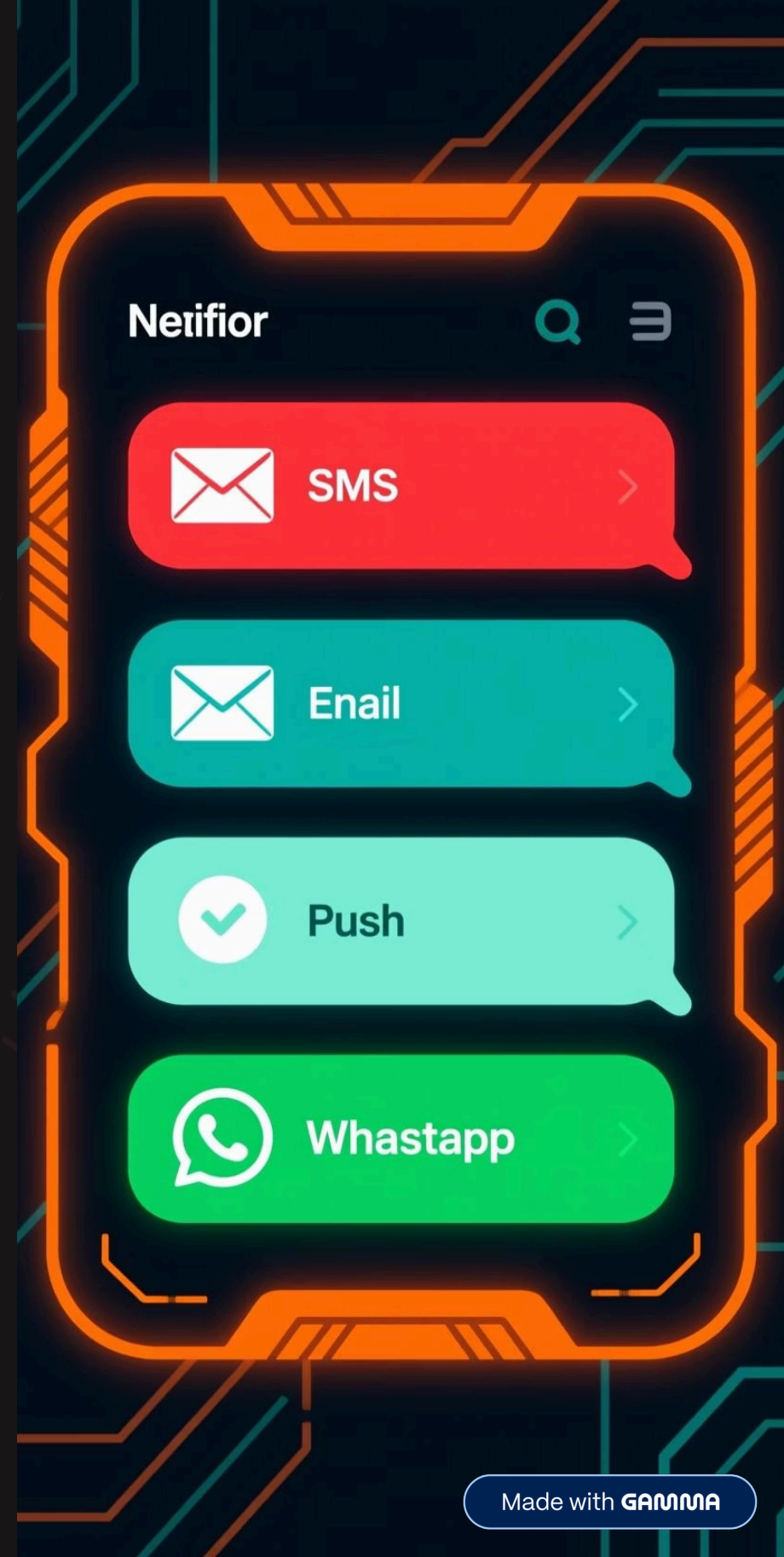
TrustNotify

Intelligent Notification Delivery System

Presented by: Avinash Pandey

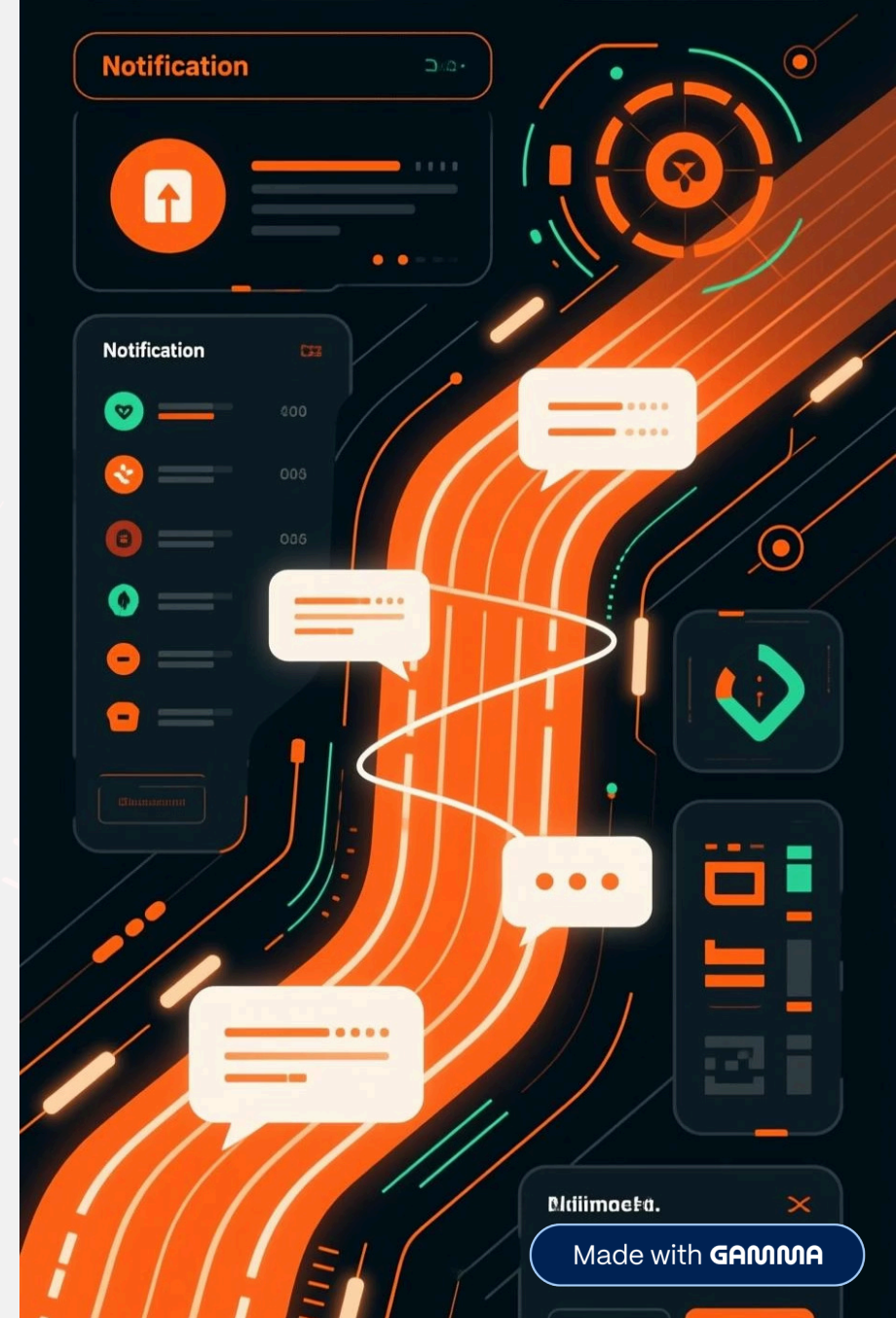
Cotton University

Provided by HDFC for Capstone Project



TrustNotify: Intelligent Notification Delivery System

A comprehensive Streamlit-based platform that combines AI-powered channel selection, real-time spam detection, and intelligent fallback mechanisms to ensure secure, reliable notification delivery across multiple channels.

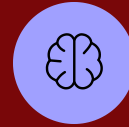


Architecture & Core Components



PostgreSQL Backend

Persistent storage with notification_log table tracking deliveries, retries, and spam detection results in real-time



AI Channel Selector

Machine learning algorithm analyzes historical delivery rates and event types to optimize channel selection automatically



Spam Detection Engine

Multi-factor scoring system monitors frequency, repeat events, and multi-channel abuse to block malicious traffic



Fallback Mechanism

Intelligent retry system ensures high-priority notifications reach users through alternative channels when primary delivery fails

From Excel to Real-Time Database

Source Data Structure

The system ingests historical notification data from **Merged_By_Event_Type.xlsx**, containing delivery metrics, retry percentages, and channel performance across multiple event types.

Key Fields Processed:

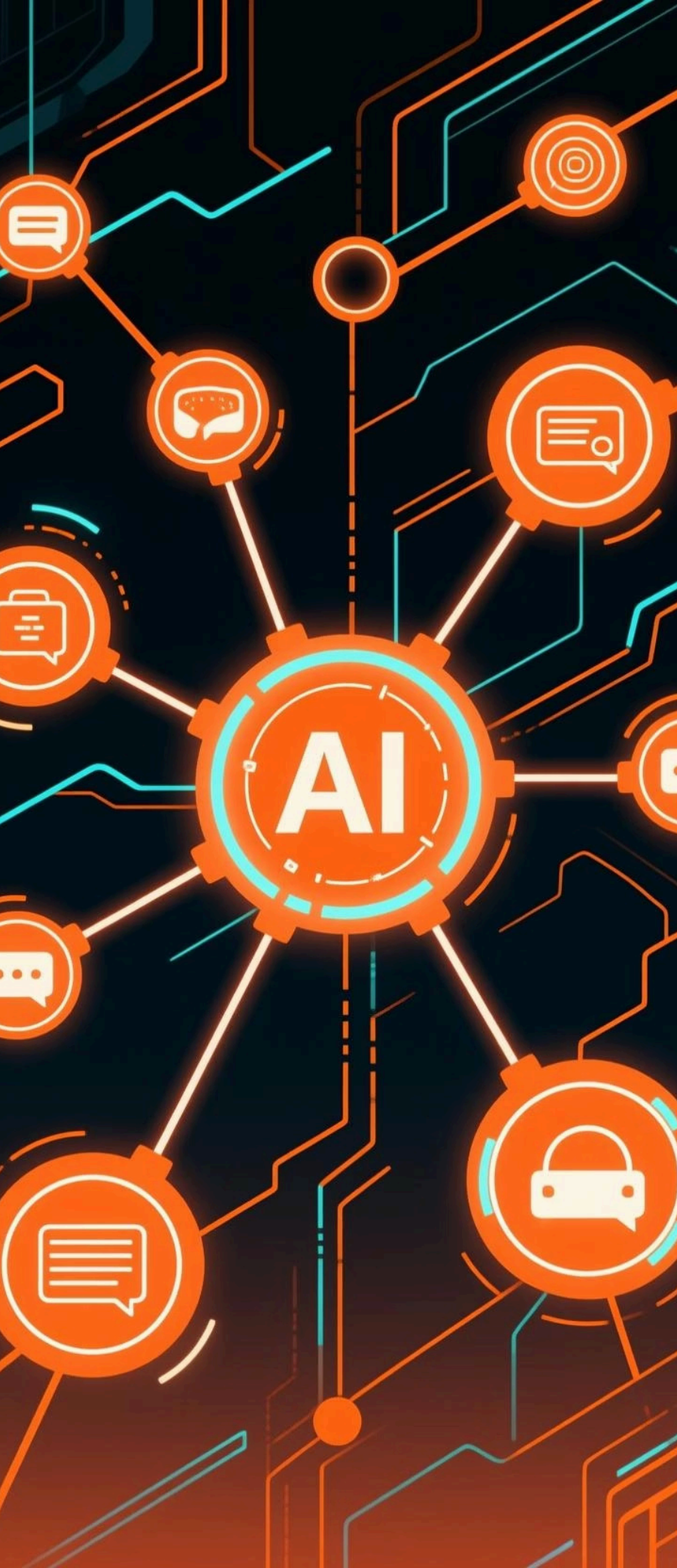
- Delivered Y/N flag converted to binary
- Retry percentage and event counts
- Intended channel assignments
- Event type classifications

Database Schema

The notification_log table captures complete audit trails with eight essential fields tracking every notification lifecycle stage.

notification_log:

- id (SERIAL PRIMARY KEY)
- timestamp
- customer_id
- event_type
- channel
- status
- retries
- spam_score
- spam_status

A central orange gear-like circle with the letters 'AI' in white is connected by lines to several other orange circles. Each of these surrounding circles contains a white icon: a speech bubble, a target, a document with a checkmark, a speech bubble with a checkmark, a speech bubble with a list, a speech bubble with a checkmark, a speech bubble with a checkmark, and a speech bubble with a checkmark. The background is dark blue with orange and white circuit-like lines.

Smart Channel Selection Algorithm

The AI-powered channel selector evaluates four delivery options—**SMS**, **Email**, **Push**, and **WhatsApp**—using a sophisticated scoring system that weighs historical delivery rates, retry frequencies, event sensitivity, and time-of-day factors.

01

Historical Analysis

Calculate baseline delivery success and retry rates from past events

02

Priority Weighting

Boost scores for OTP and fraud alerts on SMS/WhatsApp (+15 points)

03

Time Optimization

Adjust scores based on current hour (penalize email after 10 PM)

04

Channel Selection

Return highest-scoring channel with full transparency via sidebar display

Multi-Factor Spam Detection



Frequency Analysis

+30 points if customer exceeds 5 notifications



Event Repetition

+25 points for 3+ identical events from same user



Channel Abuse

+20 points when using more than 3 different channels



Sensitive Spam

+30 points for suspicious OTP or fraud alert patterns

ALLOW

Score: 0-39

Normal delivery proceeds

WARNING

Score: 40-69

Flagged for monitoring

BLOCKED

Score: 70+

Delivery prevented entirely



Intelligent Retry & Fallback Strategy

1

Primary Delivery Attempt

Notification sent via AI-selected channel with dynamic failure rate calculation (adjusted by channel reliability)

2

Failure Detection

System identifies delivery failure based on historical success rates and random variance ($\pm 5\%$)

3

Priority Assessment

High-priority events (OTP, fraud alerts, KYC reminders) trigger immediate fallback; others may retry on same channel

4

Alternative Channel Selection

Fallback engine randomly selects from remaining channels, excluding the failed primary option



WhatsApp Advantage: The system reduces failure rates by **30%** for WhatsApp deliveries, reflecting its superior reliability in real-world testing.

User Interface

Six Powerful Dashboard Views

Command Center

Real-time KPIs with event filtering, delivery rates, and live feed of recent notifications

Simulator

Test notification delivery with any event type, customer ID, and channel combination

SLA Reports

Track compliance against targets: 99% delivery, $\leq 5\%$ retry rate, $\leq 1\%$ spam blocks

Database Logs

Direct SQL query results showing complete notification history from PostgreSQL

Spam Monitor

Isolated view of flagged and blocked notifications with ALLOW, WARNING, and BLOCKED threat assessment details

Raw Data

Full access to source Excel dataset with all original fields and calculations

Key Performance Metrics

4

Delivery Channels

SMS, Email, Push Notifications, and WhatsApp supported with intelligent routing

99%

SLA Target

Delivery success rate goal maintained through fallback mechanisms

70

Block Threshold

Spam score at which notifications are automatically prevented

30%

WhatsApp Boost

Failure rate reduction when using WhatsApp as delivery channel

Technology Stack

- **Frontend:** Streamlit with custom CSS styling
- **Backend:** PostgreSQL with SQLAlchemy ORM
- **Data Processing:** Pandas and NumPy
- **Visualization:** Matplotlib for charts





Session Management

Real-Time Logging & State Persistence

In-Memory Event Log

Streamlit session state maintains a live DataFrame tracking all notifications during the current session, enabling instant dashboard updates without database queries.

Dual-Write Strategy

Every notification event triggers two parallel writes: immediate append to `session_state.event_log` and persistent INSERT into PostgreSQL `notification_log` table.

Connection Pooling

Database connections are opened and closed per operation using `psycopg2` direct connections for writes and SQLAlchemy engine for bulk reads, ensuring resource efficiency.

Extension Points for Developers

Key Configuration Variables

EXCEL_FILE: Source data path
DB_CONFIG: PostgreSQL credentials
SQLALCHEMY_URL: Connection string

Environment Variables:

- Set DB_CONFIG["password"]
- Update file paths for deployment
- Configure port numbers

Extensibility Opportunities

- **Add New Channels:** Extend channel list in `smart_channel_selector()` and update scoring logic
- **Custom Spam Rules:** Modify `spam_detection_engine()` thresholds and add new detection factors
- **Enhanced Fallback:** Implement weighted fallback selection instead of random choice
- **API Integration:** Replace `simulate()` with actual SMS/Email gateway API calls



Database Schema Updates

Modify `ensure_table_exists()` to add new columns or indexes for expanded tracking



Machine Learning Models

Replace rule-based channel selection with trained models using historical delivery data



Real-Time Monitoring

Integrate Prometheus metrics or add webhooks for external alerting systems